



POLITECNICO

MILANO 1863

Progetto di Reti Logiche

2023-2024

Prof. Fornaciari William

Cardenas Estrada Morena Caterina

Codice Persona: 10797131 Matricola: 984612

Daniel Carrozzo

Codice Persona: 10768311 Matricola: 981985

Indice

INTRODUZIONE	3
SCOPO DEL PROGETTO	3
SPECIFICHE GENERALI	3
INTERFACCIA DEL COMPONENTE	4
ARCHITETTURA	5
SCELTE IMPLEMENTATIVE	5
STATI DELLA FSM	9
RISULTATI SPERIMENTALI.....	12
CONCLUSIONI	15
RISULTATI DELLA SINTESI.....	15
OTTIMIZZAZIONI	15

Introduzione

Scopo del Progetto

Lo scopo del progetto è l'implementazione di un componente HW (descritto in VHDL) che si interfaccia con una memoria per leggere una sequenza di K parole W contenute ogni 2 celle di memoria a partire da un indirizzo dato ADD. Le parole W hanno un valore compreso tra 0 e 255, a ciascuna di esse seguono celle settate a 0.

Il componente dovrà eseguire le azioni in seguito descritte, quando necessarie:

1. Se il primo dato W è pari a 0 nessuna elaborazione avrà inizio fino a quando non si avrà una W diversa da 0;
 2. Letta una generica W diversa da 0 questa resterà invariata ma la cella di memoria successiva ad essa andrà ad essere sovrascritta con il valore di credibilità C 31;
 3. Letta una generica W pari a 0, essa dovrà essere sovrascritta con l'ultimo dato W letto diverso da 0 e la cella ad essa successiva andrà ad essere sovrascritta con l'ultimo valore di credibilità C scritto sottratto di 1;
- NB. Giunti a 0 il valore di credibilità C non verrà ulteriormente decrementato.

Specifiche Generali

Il modulo hardware da implementare è costituito da 3 ingressi primari START a 1 bit, ADD a 16 bit, K a 8 bit e da una uscita primaria DONE a 1 bit. Inoltre è caratterizzato da un segnale di CLOCK e uno di RESET, quest'ultimo è l'unico segnale asincrono in tutto il sistema.

Il modulo ha inizio una volta ricevuto il segnale di RESET, in questa fase l'uscita DONE dovrà essere 0. Quando il segnale di START sarà 1, sugli ingressi ADD e K verranno posti rispettivamente l'indirizzo di partenza e il numero di parole W da elaborare secondo le specifiche elencate sopra. Giunti alla fine dell'elaborazione il modulo dovrà alzare il segnale di DONE a 1 e mantenerlo alto fino a quando un altro segnale di START verrà posto a 1 e avrà inizio un'altra elaborazione.

Interfaccia del componente

Il componente ha la seguente interfaccia:

```
entity project_reti_logiche is
    port (
        i_clk      : in std_logic;
        i_rst      : in std_logic;
        i_start    : in std_logic;
        i_add      : in std_logic_vector(15 downto 0);
        i_k        : in std_logic_vector(9 downto 0);

        o_done     : out std_logic;

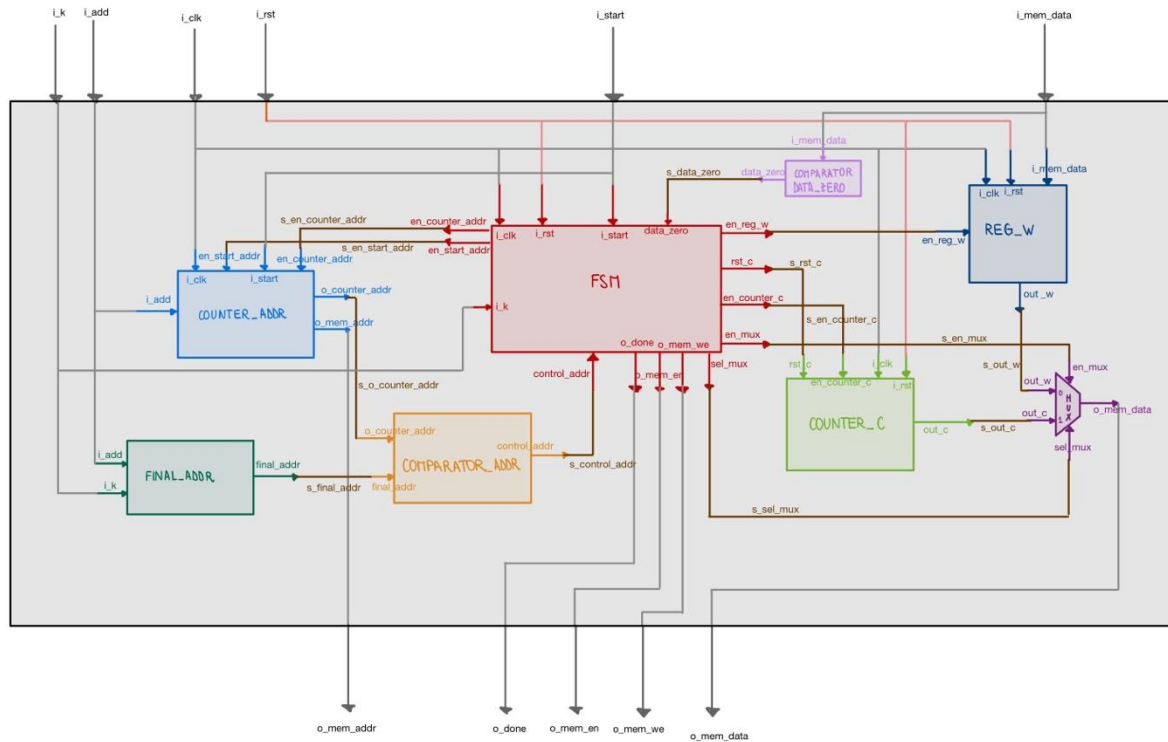
        o_mem_addr : out std_logic_vector(15 downto 0);
        i_mem_data : in std_logic_vector(7 downto 0);
        o_mem_data : out std_logic_vector(7 downto 0);
        o_mem_we   : out std_logic;
        o_mem_en   : out std_logic
    );
end project_reti_logiche;
```

In particolare:

- i_clk è il segnale di CLOCK in ingresso generato dal Test Bench;
- i_rst è il segnale di RESET che inizializza la macchina pronta per ricevere il primo segnale di START;
- i_start è il segnale di START generato dal Test Bench;
- i_k è il segnale (vettore) K generato dal Test Bench rappresentante la lunghezza della sequenza;
- i_add è il segnale (vettore) ADD generato dal Test Bench che rappresenta l'indirizzo dal quale parte la sequenza da elaborare;
- o_done è il segnale DONE di uscita che comunica la fine dell'elaborazione;
- o_mem_addr è il segnale (vettore) di uscita che manda l'indirizzo alla memoria;
- i_mem_data è il segnale (vettore) che arriva dalla memoria e contiene il dato in seguito ad una richiesta di lettura;
- o_mem_data è il segnale (vettore) che va verso la memoria e contiene il dato che verrà successivamente scritto;
- o_mem_en è il segnale di ENABLE da dover mandare alla memoria per poter comunicare (sia in lettura che in scrittura);
- o_mem_we è il segnale di WRITE ENABLE da dover mandare alla memoria (=1) per poterci scrivere. Per leggere da memoria esso deve essere 0.

Architettura

Scelte Implementative



- **COMPARATOR_ADDR (DA MODIFICARE NEL CODICE)**

Il componente ha i seguenti segnali di INPUT e di OUTPUT:

- IN a 16 bit *o_counter_addr*
- IN a 16 bit *final_addr*
- OUT a 1 bit *control_addr*

Confronta l'indirizzo a cui si accederà in memoria (*o_counter_addr*) dato dal COUNTER_ADDR con quello che determina la fine della sequenza (*final_addr*) dato da FINAL_ADDR. Nel caso in cui siano uguali ritorna un segnale (*control_addr*) a 1 altrimenti a 0.

- **COMPARATOR_DATA_ZERO**

Il componente ha i seguenti segnali di INPUT e di OUTPUT:

- IN a 8 bit *i_mem_data*
- OUT a 1 bit *data_zero*

Confronta che il dato in ingresso (i_mem_data) nella FSM proveniente dalla RAM sia uguale a 0 e in caso affermativo avvisa la FSM tramite un segnale (data_zero) posto a 1, altrimenti a 0.

- COUNTER_ADDR

Il componente ha i seguenti segnali di INPUT e di OUTPUT e SEGNALI interni:

- IN a 1 bit i_clk
- IN a 1 bit i_start
- IN a 1 bit en_start_addr
- IN a 1 bit en_counter_addr
- IN a 16 bit i_add_reg (DA MODIFICARE)
- OUT a 16 bit o_counter_addr
- OUT a 16 bit o_mem_addr
- SIG a 16 bit temp_addr

Contiene un indirizzo interno (temp_addr) inizializzato al valore del segnale di ingresso (i_add_reg) quando il segnale di avvio al componente generale (i_start) diventa pari a 1 e il segnale di inizializzazione (en_start_addr) dalla FSM diventa pari a 1. Questo indirizzo (temp_addr) aumenta di 1 ad ogni ciclo di clock (i_clk) ogni volta che riceve il segnale di incremento (en_counter_addr) proveniente dalla FSM con valore pari a 1 mentre il segnale di avvio al componente generale (i_start) è pari a 1. Dal componente esce l'indirizzo (o_counter_addr) che va al COMPARATOR_ADDR e l'indirizzo (o_mem_addr) che va alla RAM; questi due indirizzi sono uguali e rappresentano l'indirizzo di memoria corrente, sono solo divisi per praticità.

- COUNTER_C

Il componente ha i seguenti segnali di INPUT e di OUTPUT e SEGNALI interni:

- IN a 1 bit i_clk
- IN a 1 bit i_rst
- IN a 1 bit rst_c
- IN a 1 bit en_counter_c
- OUT a 8 bit out_c
- SIG a 8 bit temp_c

Tiene il conteggio del valore di credibilità (temp_c) che può essere resettato a 31 da un segnale (en_counter_c) della FSM e dal segnale di reset del componente generale (i_rst). Questo valore di credibilità diminuisce di 1 ad ogni ciclo di clock (i_clk) ogni volta che riceve il segnale (en_counter_c) proveniente dalla FSM. Se il valore di credibilità (temp_c) è pari a 0 rimane a 0. Il valore di credibilità (temp_c) viene fatto fuoriuscire tramite un segnale (out_c) e messo in ingresso nel MUX.

- FINAL_ADDR

Il componente ha i seguenti segnali di INPUT e di OUTPUT:

- IN a 16 bit i_add
- IN a 10 bit i_k
- OUT a 16 bit $final_addr$

Calcola l'indirizzo finale a cui bisogna fermarsi e lo mette in fuoriuscita in un segnale ($final_addr$) diretto a COMPARATOR_ADDR, esso è equivalente alla formula composta dai seguenti segnali in entrata $(i_add) + ((i_k) * 2)$

- FSM

Il componente ha i seguenti segnali di INPUT e di OUTPUT:

- IN a 1 bit i_start
- IN a 1 bit i_clk
- IN a 1 bit i_rst
- OUT a 1 bit en_start_addr
- OUT a 1 bit $en_counter_addr$
- IN a 1 bit $control_addr$
- OUT a 1 bit en_reg_w
- OUT a 1 bit rst_c
- OUT a 1 bit $en_counter_c$
- OUT a 1 bit en_mux
- OUT a 1 bit sel_mux
- IN a 1 bit $data_zero$
- OUT a 1 bit o_mem_en
- OUT a 1 bit o_mem_we
- OUT a 1 bit o_done

Determina la macchina a stati spiegata nello schema successivo.

- MUX

Il componente ha i seguenti segnali di INPUT e di OUTPUT:

- IN a 1 bit sel_mux
- IN a 1 bit en_mux
- IN a 8 bit out_w
- IN a 8 bit out_c
- OUT a 8 bit o_mem_data

Prende in ingresso il segnale (out_w) proveniente dal REG_W e il segnale (out_c) proveniente dal COUNTER_C. Si abilita tramite il segnale (en_mux) proveniente dalla FSM e decide quale ingresso utilizzare per metterlo in uscita sul segnale (o_mem_data) diretto alla RAM tramite il segnale sel_mux proveniente dalla FSM.

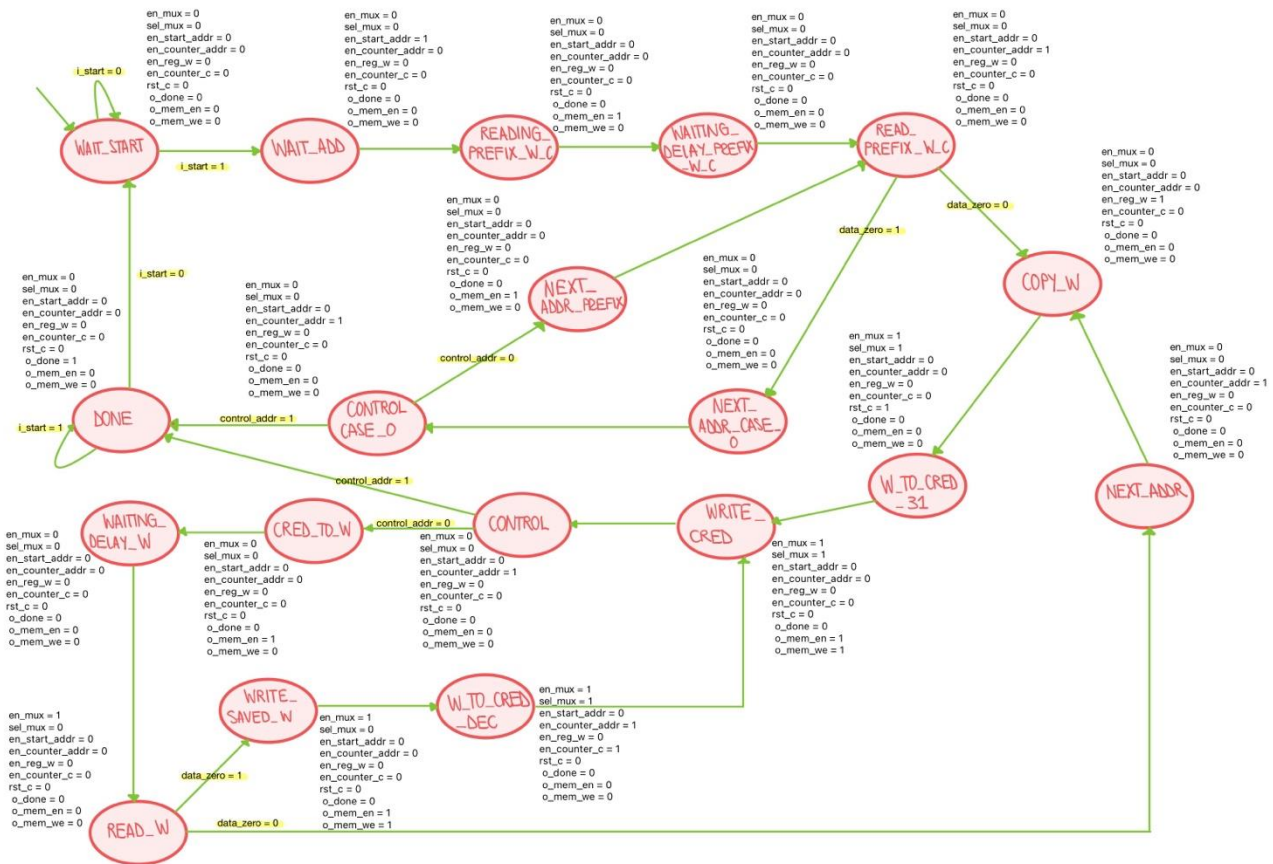
- REG_W

Il componente ha i seguenti segnali di INPUT e di OUTPUT e SEGNALI interni:

- IN a 1 bit i_rst
- IN a 1 bit i_clk
- IN a 1 bit en_reg_w
- IN a 8 bit i_mem_data
- OUT a 8 bit out_w

Si occupa di memorizzare il dato in ingresso (i_mem_data) dalla RAM. Esso viene salvato ogni segnale di clock (i_clk) quando arriva un segnale di enable (en_reg_w) dalla FSM e viene resettato quando arriva il segnale di reset del componente generale (i_rst). Il dato (out_w) viene fatto fuoriuscire e messo in ingresso nel MUX. Al segnale di reset del componente generale (i_rst) esso viene posto a 0.

Stati della FSM



Ad ogni ciclo di clock (i_clk) l'FSM cambia stato. Qual ora ci siano delle condizioni di scelta lo stato può avere diversi stati successivi dipendenti da esse. I segnali impostati dalla FSM si devono interpretare solo per quello stato e non sono mantenuti. Nel caso in cui il segnale di reset asincrono (i_rst) sia pari a 1 lo stato attuale diventa [WAIT_START].

- [WAIT_START]

Aspetta che il segnale di inizio del componente generale (i_start) sia pari a 1, quando è tale va in [WAIT_ADD].

- [WAIT_ADD]

Inizializza il contatore all'indirizzo di memoria a quello in ingresso al componente generale inviando un segnale (en_start_addr) al COUNTER_ADDR. Poi va in [READING_PREFIX_W_C].

- [READING_PREFIX_W_C]

Stato ponte per permettere l'abilitazione alla lettura in memoria tramite un segnale (o_mem_en) diretto alla RAM con l'indirizzo di partenza. Poi va in [WAITING_DELAY_PREFIX_W_C].

- [WAITING_DELAY_PREFIX_W_C]

Stato ponte per permettere che il dato arrivi in uscita dalla RAM. Poi va in [READ_PREFIX_W_C].

- [READ_PREFIX_W_C]

Imposta il segnale (en_counter_addr) diretto verso il COUNTER_ADDR a 1 passando all'indirizzo successivo. Nel caso in cui il segnale (data_zero) proveniente dal COMPARATOR_DATA_ZERO sia pari a 1 inizia il ciclo per controllare la sequenza di zeri iniziale andando in [NEXT_ADDR_CASE_0], altrimenti inizia l'algoritmo classico andando in [COPY_W].

- [NEXT_ADDR_CASE_0]

Stato ponte che salta lo 0 nella cella successiva a W nella sequenza di zeri iniziale. Poi va in [CONTROL_CASE_0].

- [CONTROL_CASE_0]

Imposta il segnale (en_counter_addr) diretto verso il COUNTER_ADDR a 1 passando all'indirizzo successivo. Nel caso in cui il segnale (control_addr) che indica l'arrivo all'ultimo indirizzo proveniente dal CONTROL_ADDR è pari a 1, la sequenza si ritiene conclusa e va in [DONE], altrimenti continua e va in [NEXT_ADDR_PREFIX].

- [NEXT_ADDR_PREFIX]

Stato ponte per permettere l'abilitazione alla lettura in memoria tramite un segnale (o_mem_en) diretto alla RAM con l'indirizzo attuale. Poi va in [READ_PREFIX_W_C]

- [COPY_W]

Imposta il segnale (en_reg_w) diretto a REG_W a 1 per salvare la W. Poi va in [W_TO_CRED_31].

- [W_TO_CRED_31]

Resetta il valore di credibilità tramite un segnale (rst_c) in uscita verso il COUNTER_C e imposta già il MUX abilitandolo con il segnale (en_mux) e selezionando l'uscita (sel_mux) a 1 per scrivere il valore di credibilità. Poi va in [WRITE_CRED].

- [WRITE_CRED]

Scriva il valore di credibilità mantenendo il MUX abilitato con il segnale (en_mux) e l'uscita selezionata (sel_mux) a 1 per scrivere il valore di credibilità, per farlo abilita i segnali di lettura (o_mem_en) e scrittura (o_mem_we) verso la RAM. Poi va in [CONTROL].

- [CONTROL]

Imposta il segnale (en_counter_addr) diretto verso il COUNTER_ADDR a 1 passando all'indirizzo successivo. Nel caso in cui il segnale (control_addr) che indica l'arrivo

all'ultimo indirizzo proveniente dal CONTROL_ADDR è pari a 1, la sequenza si ritiene conclusa e va in [DONE], altrimenti continua e va in [CRED_TO_W].

- [CRED_TO_W]

Stato ponte per permettere l'abilitazione alla lettura in memoria tramite un segnale (o_mem_en) diretto alla RAM con l'indirizzo attuale. Poi va in [WAITING_DELAY_W]

- [WAITING_DELAY_W]

Stato ponte per permettere che il dato arrivi in uscita dalla RAM. Poi va in [READ_W].

- [READ_W]

Imposta già il MUX abilitandolo con il segnale (en_mux) e selezionando l'uscita (sel_mux) a 0 per scrivere il valore di W salvato qualora necessario. Nel caso in cui il segnale (data_zero) proveniente dal COMPARATOR_DATA_ZERO sia pari a 1 procede con lo scrivere la W salvata andando in [WRITE_SAVED_W], altrimenti procede con il salvarsi W andando in [NEXT_ADDR].

- [NEXT_ADDR]

Imposta il segnale (en_counter_addr) diretto verso il COUNTER_ADDR a 1 passando all'indirizzo successivo. Poi va in [COPY_W].

- [WRITE_SAVED_W]

Scrive il valore di W salvato mantenendo il MUX abilitato con il segnale (en_mux) e l'uscita selezionata (sel_mux) a 0 per scrivere la W salvata, per farlo abilita i segnali di lettura (o_mem_en) e scrittura (o_mem_we) verso la RAM. Poi va in [W_TO_CRED_DEC].

- [W_TO_CRED_DEC]

Imposta il segnale (en_counter_addr) diretto verso il COUNTER_ADDR a 1 passando all'indirizzo successivo. Decrementa il valore di credibilità tramite un segnale (en_counter_c) in uscita verso il COUNTER_C e imposta già il MUX abilitandolo con il segnale (en_mux) e selezionando l'uscita (sel_mux) a 1 per scrivere il valore di credibilità. Poi va in [WRITE_CRED].

- [DONE]

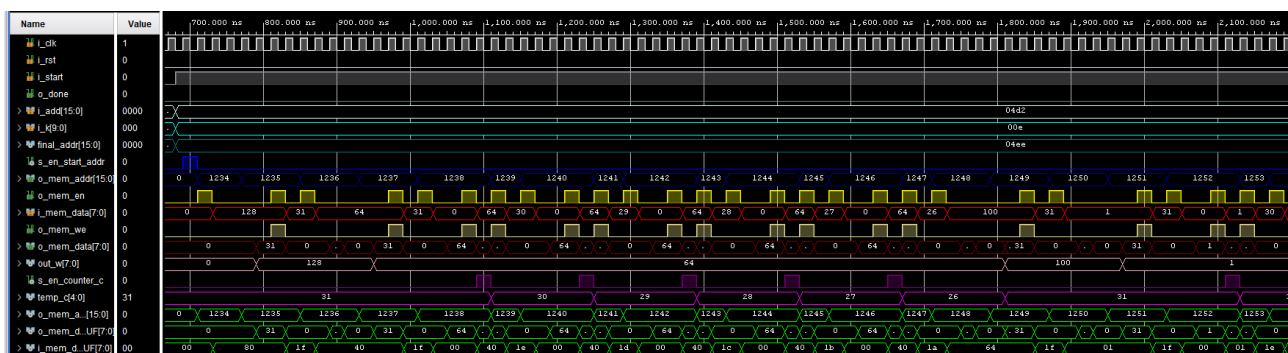
Imposta il segnale (i_done) diretto verso l'esterno del componente generale a 1. Aspetta che il segnale di inizio del componente generale (i_start) sia pari a 0, quando è tale va in [WAIT_START].

Risultati Sperimentali

Per testare il corretto funzionamento del nostro componente abbiamo utilizzato il Test Bench dato da specifica e ne abbiamo creati altri, riportiamo in seguito i test più significativi il cui obbiettivo era testare il comportamento del componente in casi limite.

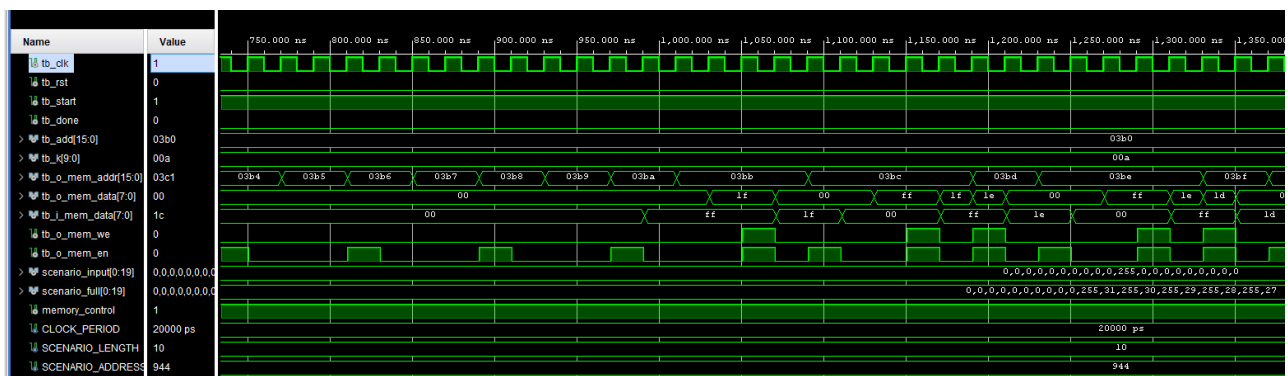
Test Bench dato da specifica

Simula il comportamento del componente per una generica sequenza senza testare particolari casi limiti ma il corretto funzionamento.



Test Bench 1

Utilizzato per testare il corretto funzionamento del componente data una sequenza di soli 0 seguita da una generica sequenza.



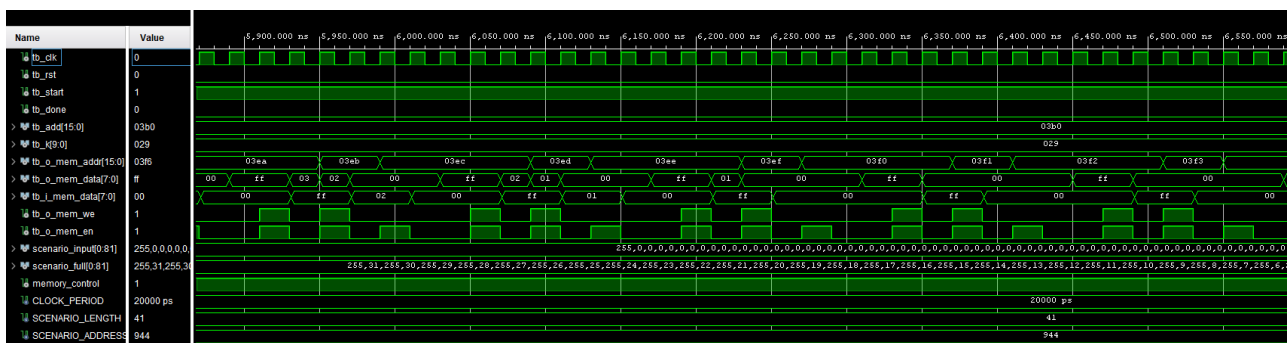
Test Bench 2

Utilizzato per testare il corretto funzionamento del componente data una sequenza di soli 0.



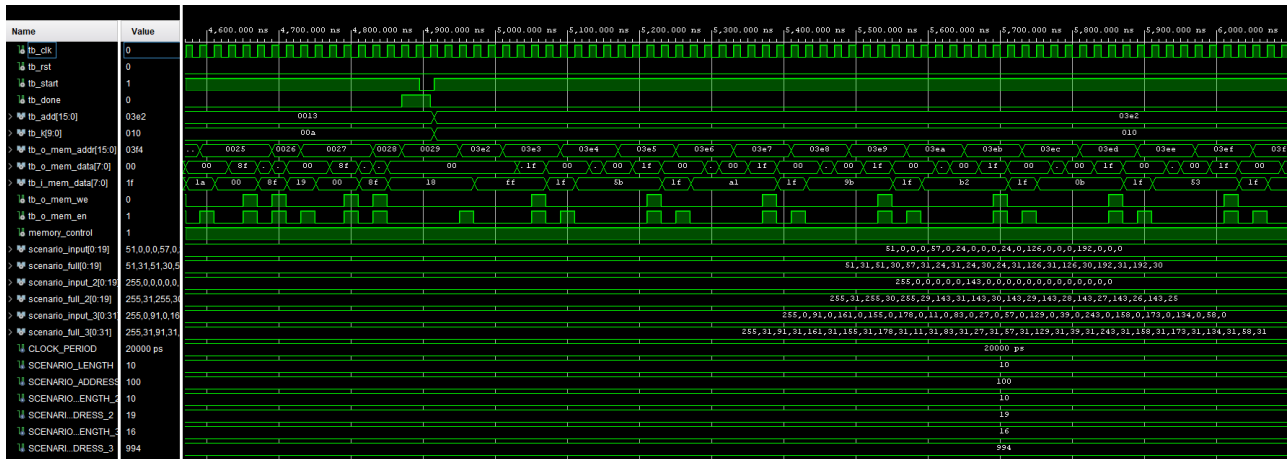
Test Bench 3

Utilizzato per testare il corretto funzionamento del componente data una W diversa da 0 e una lunga sequenza di W pari a 0. In questo modo siamo andati a testare la funzione di decremento del valore di credibilità C, in particolare quando questo raggiungeva il valore 0.



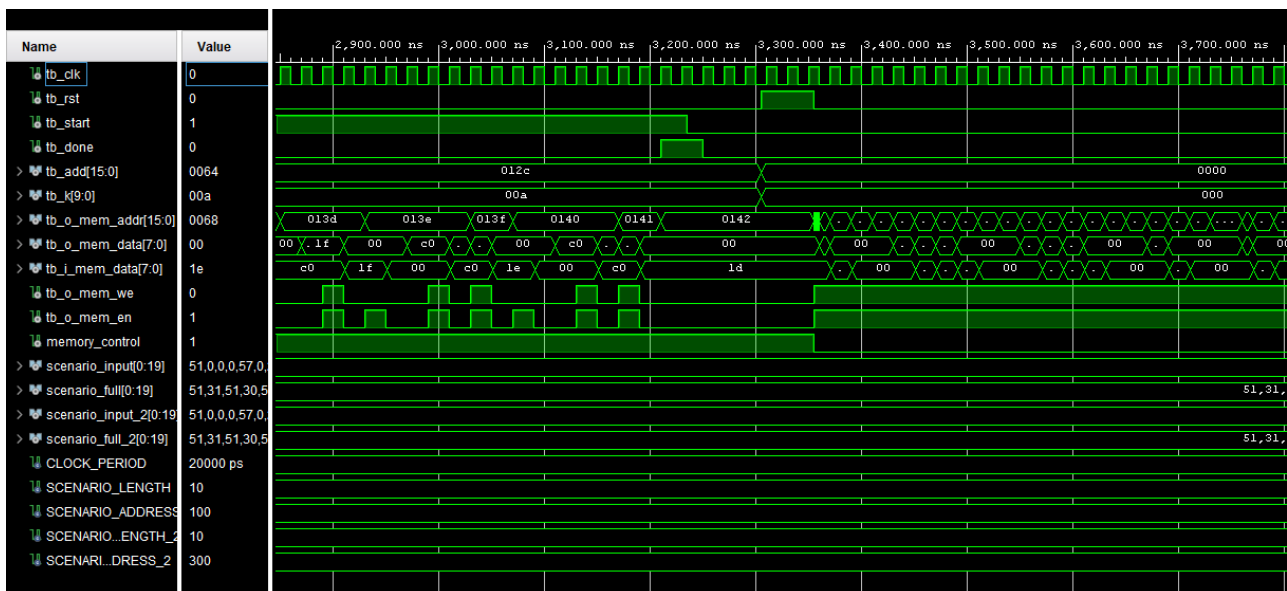
Test Bench 4

Utilizzato per testare il corretto funzionamento del componente nell'elaborazione di più sequenze, in questo caso siamo andati a verificare che i segnali di DONE e START funzionassero correttamente.



Test Bench 5

Utilizzato per testare il corretto funzionamento del componente in particolare quando si riceve il segnale di RST durante l'esecuzione.



Conclusioni

Risultati della Sintesi

Il componente sintetizza correttamente tutti i test precedentemente descritti in tutte e 3 le simulazioni richieste: *Behavioural*, *Synthesis* e *Post-Synthesis Functional*.

Timing Report

```
Slack (MET) :          16.549ns  (required time - arrival time)
Source:        counter_addr_1/temp_addr_reg[0]/C
                (rising edge-triggered cell FDRE clocked by c
Destination:   fsm_1/FSM_onehot_curr_state_reg[12]/D
                (rising edge-triggered cell FDCE clocked by c
```

In particolare nella Post-Synthesis abbiamo verificato tramite il comando *report_utilization* l'assenza di Latch, come previsto.

Site Type	Used	Fixed	Prohibited	Available	Util%
Slice LUTs*	59	0	0	8000	0.74
LUT as Logic	59	0	0	8000	0.74
LUT as Memory	0	0	0	5000	0.00
Slice Registers	49	0	0	16000	0.31
Register as Flip Flop	49	0	0	16000	0.31
Register as Latch	0	0	0	16000	0.00
F7 Muxes	0	0	0	7300	0.00
F8 Muxes	0	0	0	3650	0.00

Inoltre il componente soddisfa i requisiti di tempo che abbiamo verificato utilizzando il comando *report_timing*.

Ottimizzazioni

Le ottimizzazioni che si potrebbero fare riguardo la FSM sono in particolare la riduzione del numero degli stati che la compongono, dato che molti di questi sono “Stati Ponte”. Essi sono stati utilizzati esclusivamente per aspettare un ciclo di clock.

Inoltre, si potrebbero ridurre i componenti per il calcolo dell'indirizzo ma abbiamo effettuato una scelta implementativa tale da permettere modifiche e salvataggi più agevoli dell'indirizzo corrente nel caso di modifiche del componente in futuro.